Introduction to Redis

The ultimate solution for all your caching needs... and more!

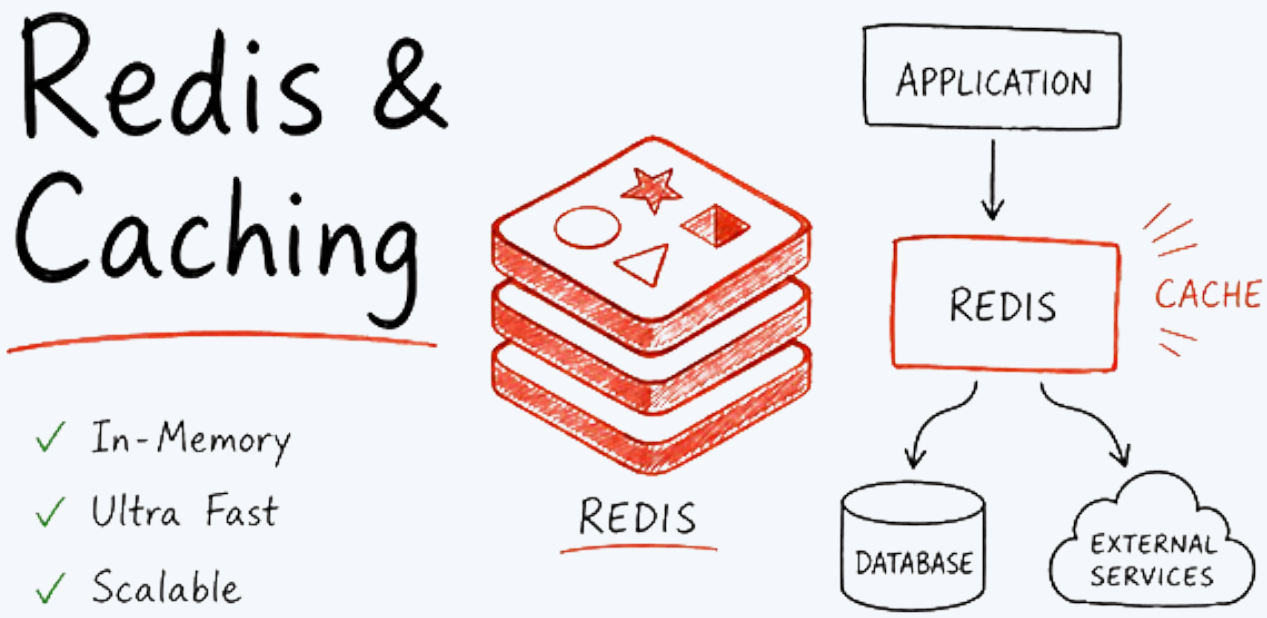
Supporting scalibility since 7 seconds ago...

What is Redis?

- In - memory database
  - Very fast!
- Supports many different datatypes!
  - Hashes
  - Lists
  - Sorted Lists
  - Sets
  - Streams
- Pub Sub
  - One Publisher, Multiple Subscribers

Why should I use Redis?

- Scalibility
  - Shared across pods
  - Unlike in-pod data structures
  - OKD!
- Very fast cache!
  - Good for standing infront of databases!
- Needing a TTL (Time to Live) for caching
  - Stampeding Herd issue.
- Need to send information to many things
  - Exact use of pub sub
    - Can use another hash for late joins
- Good for Rate Limiting



When should I use Redis?

- Storage in Multi-pod architecture
  - OKD
- Need Time to Live for items
- Leaderboards
- Multiple clients all seing Live data
- Faster response times from databases
  - Indices?

Why doesn't everyone use Redis?!?!

- Introduces New Issues / Bugs to Solve
  - Memory Leaks
  - Cache Busting
  - Thundering Herd / Stampeding Herd
  - Scripting Issues
  - Deadlocks
- System Complexity
  - Sharding
  - Routing
- Over-Engineering
  - Yes, this is actually valid.

QUESTIONS?

Any questions before showing examples?

SECTION

Real Redis Examples

Examples of Redis Usecases in the real world, using the Golang SDK.

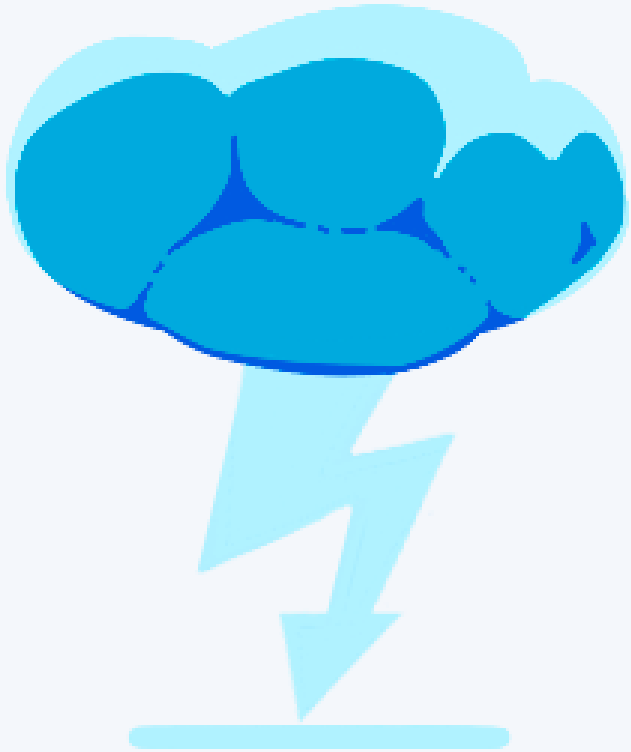
SECTION

EX 1: Basic Redis Usage

Multi-Pod Redis Caching with Locks, used in Store 'em Cloud (storemcloud.cs.house)

Purpose

- S3 requires presigned Urls, but a new one is generated every time, which ruins browser caching!
  - Seperate rabbit hole. for a different time.
- We need to save generated presigned urls for a specific video to help browsers cache.
- We can use a Redis cache with a TTL for this!



Connecting to a Redis Client

- Firstly, on pod startup we connect to our Redis Client.
- Best Practices
  - Always ping after connecting to make sure its a stable connection
  - Your redis creds should be environmental variables!

```go
var client *redis.Client
var RedisInitalized bool = false

func InitRedis() error {
    ctx, cancel := context.WithTimeout(context.Background(), time.Second*10)
    defer cancel()

    opt, err := redis.ParseURL(os.Getenv("REDIS_URL"))
    if err != nil {
        return err
    }

    newClient := redis.NewClient(opt)
    _, err = newClient.Ping(ctx).Result()
    if err != nil {
        _ = newClient.Close()
        return err
    }

    RedisInitalized = true
    client = newClient
    return nil
}
```

Getting with Redis

- When we are getting a presigned url, we need to first check if a value exists. If so we can just use the value!
- IF NOT, we need to “lock” the index so only one pod can mod it. WITH a TTL
- Best Practices
  - Redis keys should follow a thing:thing format.
    - **presignedurl:video:videoid** in here.
  - Give your locks a short TTL to avoid deadlocking!

```go
func GetPresignedURL(s3Key string, ver URLTypes) (string, error) {
    if !RedisInitalized {
        return "", errors.New("redis was not intialized")
    }
    ctx, cancel := context.WithTimeout(context.Background(), time.Second*15)
    defer cancel()

    cacheKey := fmt.Sprintf("presigned:%s%s", ver, s3Key)

    url, err := client.Get(ctx, cacheKey).Result()
    if err == nil {
        return url, nil
    }
    if err != redis.Nil {
        return "", err
    }

    lockKey := fmt.Sprintf("lock:%s%s", ver, s3Key)

    setLock, err := client.SetNX(ctx, lockKey, 1, time.Second*10).Result()
    if err != nil {
        return "", err
    }
```

Setting With Redis

- Well in the case it doesn’t exist, we need to set it!
- We generate a new presigned URL, and then set it!
- Best Practices
  - Stagger your time to lives to avoid a bunch of keys resetting at once.
    - Helps against Thundering Herds.

```
if setLock {
    defer client.Del(ctx, lockKey)

    newURL, err := refreshKeyVal(s3Key, ver)
    if err != nil {
        return "", err
    }

    ttl := getRandomTTL()

    if err := client.Set(ctx, cacheKey, newURL, ttl).Err(); err != nil {
        return "", err
    }

    return newURL, nil
}
```

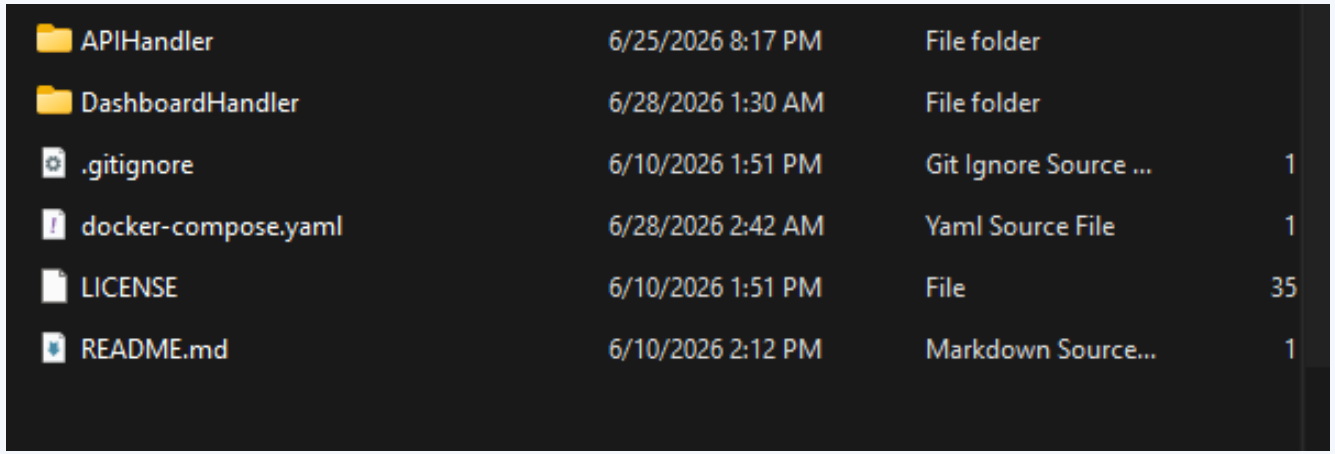
SECTION

EX 2: Redis Pub / Sub

Multi-Pod Publication / Subscription for the SERGE Storm Team using Redis

Infrastructure

- APIHandler
  - Recieves Live Info from the Storm Probe
  - Handles Database writing / info logging
  - Updates Dashboards
- DashboardHandler
  - Microservice for the Dashboard
  - Handles viewers seeing Live Data
- So How does One API pod handle 3+ Dashboard pods?
  - Pub Sub!



Setting Redis Connection for Pub Sub

- Connecting To Redis Client
  - Seperate connection for Pub Sub
- Confirming Connection before going forward.

```go
var client *redis.Client
var chanClient *redis.Client

func InitRedis() error {
    ctx, cancel := context.WithTimeout(context.Background(), time.Second*15)
    defer cancel()

    opt, err := redis.ParseURL(os.Getenv("REDIS_URL"))
    if err != nil {
        return err
    }

    newClient := redis.NewClient(opt)
    _, err = newClient.Ping(ctx).Result()
    if err != nil {
        return err
    }

    newChanClient := redis.NewClient(opt)
    _, err = newChanClient.Ping(ctx).Result()
    if err != nil {
        return err
    }

    client = newClient
    chanClient = newChanClient
    logging.Logger.WithFields(logrus.Fields{"module": "redis", "method": "InitRedis"}).Info("Successfully Initialized Redis!")
    return nil
}
```

Subscriber

- Recieves new messages when published
  - DOES NOT have a “latest” message.
  - Use a hashmap to accomplish this
- Updates all connected websocket with latest message
- Channel: “zephyr-update”

```go
func SubscribeToZephyrApi() error {
    ctx := context.Background()

    sub := chanClient.Subscribe(ctx, "zephyr-update")
    defer sub.Close()

    subChannel := sub.Channel()

    for {
        select {
        case payload, ok := <-subChannel:
            if !ok {
                return fmt.Errorf("redis subscription closed")
            }

            var update types.ZephyrUpdate
            err := json.Unmarshal([]byte(payload.Payload), &update)
            if err != nil {
                logging.Logger.WithFields(logrus.Fields{"error": err, "module": "redis", "method": "InitRedis"}).Warn("Error unmarshaling message!")
                continue
            }

            dashboard.MessageAllWebsockets(update)
        case <-ctx.Done():
            return ctx.Err()
        }
    }
}
```

Publisher

- Publishes new messages into the channel
- Same Channel: “zephyr-update”

```go
func PublishToDashboard(payload any) error {
    ctx := context.Background()

    err := chanClient.Publish(ctx, "zephyr-update", payload).Err()
    if err != nil {
        return err
    }

    return nil
}
```

Publisher Recieves and Broadcasts New Data

- Recieves data from the probe.
- Publishes new data through the previously stated Redis channel

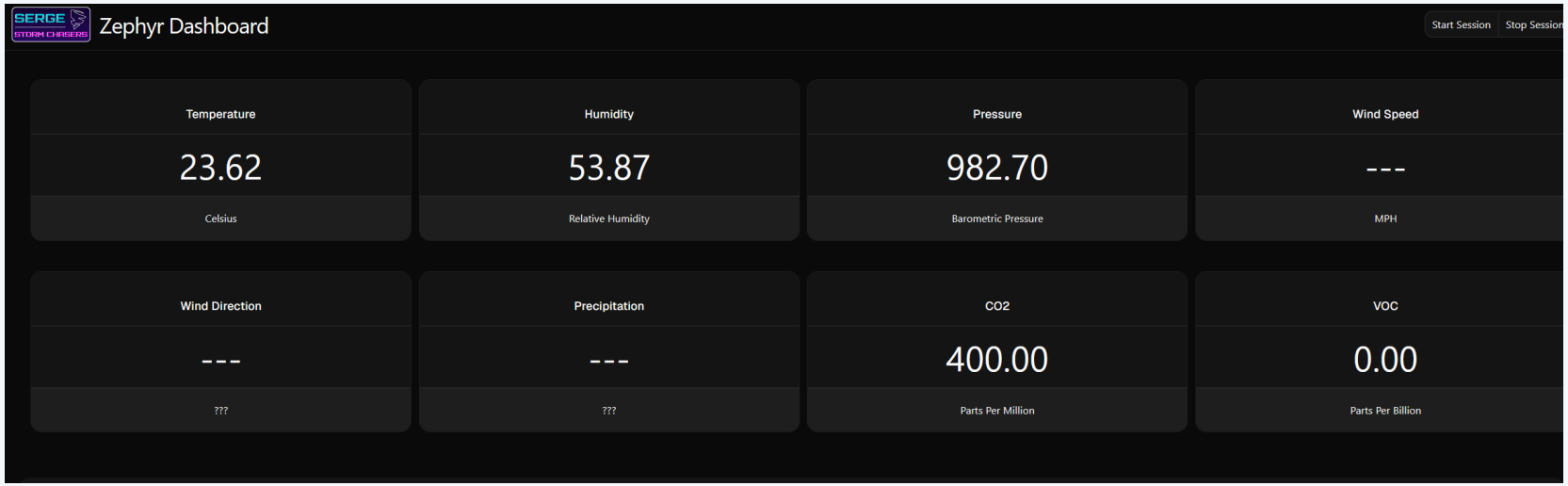
```go
func LiveData(c *gin.Context) {
    var payload types.ZephyrUpdate
    err := c.ShouldBindJSON(&payload)
    if err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Unable to publish data"})
        logging.Logger.WithFields(logrus.Fields{"error": err, "module": "api", "method": "LiveData"}).Warn("Unable to parse data")
        return
    }

    err = redis.PublishToDashboard(payload)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Unable to publish data"})
        logging.Logger.WithFields(logrus.Fields{"error": err, "module": "api", "method": "LiveData"}).Warn("Unable to publish information to websocket!")
        return
    }

    c.JSON(http.StatusOK, gin.H{
        "status": "ok",
    })
}
```

Subscriber Recieves!

- Websockets transmit the newly published data
- All viewer dashboards update live!



SECTION

EX 3: Rate Limiting With Redis

Token Bucket Strategy using Redis .lua scripts

Goal

- Rate Limiting websites is crucial!!!
- Multiple different strategies
  - Fixed Window
  - Sliding Window
  - Leaky Bucket
  - **Token Bucket**
- Persistent across Pods!

Starting with Token Bucket

- Start with a fixed number of tokens
- Each request removes one.
- After a certain duration, drop a new token into the bucket.
- If there is no tokens in the bucket, deny the request.

Uh Oh! Race Conditions!

- What happens if two different requests try to update it at the same time?
- Race Condition!
- Entire operation needs to be atomic.
  - Atomic: Everything happens at once, meaning no race conditions.
- Luckily, we can do this with .lua scripts!

Making our Token Bucket

- Firstly, we have to make our token bucket object
- Each Token Bucket will have a redis connection, a recharge rate, and the amount of tokens it can hold at once.

```go
func NewTokenBucket(rate float64, capacity float64) *TokenBucket {
    return &TokenBucket{client: client, rate: rate, capacity: capacity}
}
```

Creating a Lua Script

- Lua script runs all operations at once.
- Able to set local variables, just like standard .lua Scripting
- Can pass arguments into the script.
- Can also use all redis commands inside the script!
- Returns back to the scripts call site.

```
var middlewareScript = redis.NewScript(`
    local key = KEYS[1]
    local now = tonumber(ARGV[1])
    local rate = tonumber(ARGV[2])
    local capacity = tonumber(ARGV[3])
    local ttl = tonumber(ARGV[4])

    local bucketData = redis.call("HMGET", key, "tokens", "last_refill")
    local tokens = tonumber(bucketData[1]) or capacity
    local lastRefill = tonumber(bucketData[2]) or now

    local elapsed = math.max(0, (now - lastRefill) / 1000)
    tokens = math.min(capacity, tokens + (elapsed * rate))

    if tokens < 1 then
        redis.call("HMSET", key, "tokens", tokens, "last_refill", now)
        redis.call("PEXPIRE", key, ttl)
        return {0, tokens}
    end

    tokens = tokens - 1
    redis.call("HMSET", key, "tokens", tokens, "last_refill", now)
    redis.call("PEXPIRE", key, ttl)
    return {1, tokens}
`)
```

Attaching Our Middleware

- Using the Gin framework in go, we can attach a middleware.
- We calculate the difference in time to generate tokens.
- Determine if the user is allowed to pass or not.

```go
func (tb *TokenBucket) Allow(ctx context.Context, key string) (bool, int64, error) {
    now := time.Now().UnixMilli()
    ipKey := fmt.Sprintf("%s%s", RatelimitKey, key)
    ttlMs := int64((tb.capacity / tb.rate) * MILLISECONDS * 2)

    result, err := middlewareScript.Run(ctx, tb.client, []string{ipKey}, now, tb.rate, tb.capacity, ttlMs).Int64Slice()
    if err != nil {
        return false, 0, fmt.Errorf("redis script: %w", err)
    }

    return (result[0] == 1), result[1], nil
}

func RedisRateLimiter(rate float64, capacity float64) gin.HandlerFunc {

    if client == nil {
        logging.Logger.WithFields(logrus.Fields{"module": "api", "method": "RedisRateLimiter"}).Warn("Redis cache was not

        return func(c *gin.Context) {
            c.Next()
        }
    }

    limiter := NewTokenBucket(rate, capacity)

    return func(c *gin.Context) {
        ip := c.ClientIP()
        allowed, tokens, err := limiter.Allow(c, ip)

        if err != nil {
            logging.Logger.WithFields(logrus.Fields{"error": err, "module": "api", "method": "RedisRateLimiter"}).Warn("Fa
        } else if !allowed {
            c.JSON(http.StatusTooManyRequests, gin.H{
                "error": "Too many requests!",
            })
            c.Abort()
            return
        }

        c.Header("X-RateLimit-Limit", fmt.Sprintf("%.0f", capacity))
        c.Header("X-RateLimit-Remaining", fmt.Sprintf("%v", tokens))

        c.Next()
    }
}
```

Thank you for listening! Any questions?